Desenvolupament d'una estació de seguiment de megaconstel·lacions de satèl·lits LEO mitjançant un dispositiu RFSoC programable d'alta velocitat

Jan Moran Ricardo

29 de juny de 2026

Resum– Aquest projecte presenta el disseny i desenvolupament d'un sistema de programari per al control, abstracció i simplificació del procés de captura de senyals de megaconstel·lacions de satèl·lits LEO. Es farà ús de la ràdio definida per software (SDR) d'altes prestacions: la USRP X440. Aquest dispositiu es basa en una arquitectura de mostreig directe (Direct RF), la qual, si bé ofereix grans avantatges en el domini analògic, introdueix una elevada complexitat de configuració i tractament. Per resoldre aquest problema, s'han dissenyat una sèrie de programes, algorismes i interfícies que calculen, processen i simplifiquen el procés de captura amb l'objectiu de facilitar l'ús a futurs usuaris. Tots els sistemes dissenyats i implementats al llarg d'aquest projecte s'han validat amb èxit al laboratori tant amb senyals sintètics com amb senyals reals de la constel·lació de satèl·lits Iridium.

Paraules clau– *Software Defined Radio* (SDR), USRP X440, RFNoC, Direct RF, Full-Stack Architecture, VUE.js, Python, LEO Satellites, Signal Processing.

1 Motivació i Objectius

L'INTERÈS pel sector espacial ha experimentat un creixement exponencial en els darrers anys, impulsat principalment pels grans avenços en les tecnologies d'òrbita terrestre baixa (LEO, *Low Earth Orbit*), les quals han permès, entre d'altres, el desplegament de megaconstel·lacions com Iridium o Starlink per a les telecomunicacions globals.

És en aquest context en què entra el departament promotor d'aquest TFG, el qual, igual que molts altres centres de recerca, es troba endinsat en la busqueda de formes d'aprofitar al màxim aquestes comunicacions espacials per aplicacions civils.

El primer pas per tractar amb les comunicacions satel·litàries és captar el senyal proporcionat pels mateixos satèl·lits. La forma més popular d'aconseguir això és fent servir uns dispositius anomenats SDRs ("*Software Defined Radios*"en anglès). Aquests SDRs ens permeten captar senyals de ràdio sense la necessitat de dissenyar un *hardware* analògic específic per a cada aplicació, ja que aquests es poden assolir directament per via *software*, donant una flexibilitat immensa a l'investigador.

El nostre departament ha adquirit recentment una USRP x440 de la companyia Ettus Research [1], un now model de SDR d'altes prestacions capaç de capturar grans volums de dades de manera simultània.

Serà doncs objecte d'aquest treball estudiar, entendre i realitzar alguna aplicació pràctica amb aquest dispositiu. Concretament se'ns demana:

1. Estudiar la USRP X440 per tal d'entendre el seu funcionament i documentar totes les formes que tenim de fer-la servir.

2. Realitzar una sèrie de tests, inclòs algun de captura de senyal real, per tal de validar el seu funcionament.

3. Crear una interfície d'usuari que permeti a altres membres del departament capturar dades amb la USRP de forma senzilla.

• E-mail de contacte: jan.moran@uab.cat
• Menció: Enginyeria de Computadors
• Treball tutoritzat per: José A. Lopez Salcedo (Departament de Telecomunicació i Enginyeria de Sistemes)
• Curs 2025/26

[1] A l'estat de l'art d'aquest document es tractarà a fons aquest dispositiu.

2 MARC TEÒRIC

TOT i ser un treball de fi de grau de la carrera d'informàtica, en ell es tracten molts conceptes de matèria de telecomunicacions. És per això que en aquest marc teòric es farà un resum de les bases del tractament de senyals i sistemes per a poder entendre tota la matèria que vindrà a continuació.

2.1 Senyals

En l'àmbit de les telecomunicacions, **un senyal és qualsevol magnitud física que varia en el temps o en l'espai i que transporta informació.** Aquesta variació pot ser de diferent naturalesa, com ara la tensió elèctrica en un cable de coure, la intensitat de la llum en una fibra òptica o les ones electromagnètiques propagant-se per l'aire.

A l'hora de tractar amb senyals, una de les propietats més importants que cal tindre en compte és la freqüència. La **freqüència** es defineix com el nombre de vegades que un cicle d'un senyal periòdic es repeteix en un segon.

A l'hora de representar un senyal es pot fer o en el **domini temporal** o en el **domini freqüencial**. Per traduir una funció d'un domini a l'altre es pot fer ús de la **transformada de Fourier** (vegeu la fórmula 1). A la figura 1 queden representats aquests dos dominis.

$$X(f) = \int_{-\infty}^{\infty} x(t) e^{-j2\pi ft} dt \tag{1}$$

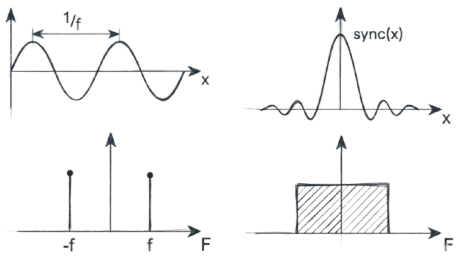


Fig. 1: Representació de dos senyals en els seus dominis temporals i freqüencials

Al conjunt total de freqüències se l'anomena **espectre**, però se sol treballar amb el concepte de **banda**: un conjunt de freqüències delimitades per una freqüència superior i inferior; i **amplada de banda**: la diferència entre ambdós límits.

Un concepte que cal tindre en compte és que quan s'envia un senyal per un medi a una freqüència determinada, aquest només es veurà afectat per senyals a la mateixa freqüència[2]. Això fa que es puguin enviar dos senyals diferents de diferents freqüències per un mateix medi i poder recuperar ambdós senyals al complet i sense interferències.

2.1.1 Conversió de freqüències (Heterodinatge)

L'**heterodinatge** és el procés de barrejar un senyal amb un altre per tal d'obtenir-ne un de nou en una freqüència diferent. Aquest procés es basa en el principi de mescla no lineal o multiplicació de senyals.

Quan es multiplica un senyal de freqüència f_1 per un senyal sinusoidal d'una altra freqüència f_2, el resultat conté components en la suma (f_1+f_2) i en la diferència ($|f_1-f_2|$) de les seves freqüències originals.

Aquesta propietat (i.e. **modulació**) és fonamental en les telecomunicacions, ens serveix per elevar o baixar senyals en freqüència per la seva emissió o recepció.

2.1.2 Senyals analògics i digitals

Segons la seva naturalesa, els senyals es divideixen en dues grans categories, cadascuna de les quals presenta els seus avantatges i inconvenients[3]:

- Senyal **Analògic**: És una funció contínua en el temps que pot prendre un rang infinit de valors dins d'un interval determinat.
- Senyal **Digital**: És una representació discreta de la informació. El senyal només pot prendre un nombre finit de valors en instants de temps específics.

2.1.3 Procés de digitalització del senyal

Com que els senyals que es troben a la natura solen ser analògics i els senyals amb què treballen els ordinadors són quasi sempre digitals, sovint s'ha de traduir entre ambdós tipus. Per passar d'un senyal analògic a un de digital s'han de realitzar les següents fases[4]:

1. Mostreig (*Sampling*): Consisteix a prendre mostres puntuals de l'amplitud del senyal analògic en intervals de temps regulars. El paràmetre que defineix aquest procés és la **freqüència de mostreig** (f_s), que indica el nombre de mostres per segon.
2. Quantificació: En aquesta fase, el valor de cada mostra s'assigna a un nivell discret dins d'una escala acabada.
3. Codificació: Finalment, cada nivell quantificat es tradueix a un codi binari que pot ser processat per l'ordinador.

Un dels punts més importants i crítics a l'hora de realitzar aquesta transformació és la tria de la freqüència de mostreig. Si es tria una freqüència massa baixa, es pedrà informació i no es podrà reconstruir el senyal de forma fidel.

Per evitar-ho, ens regim pel **Teorema de Nyquist-Shannon**, que estableix que per digitalitzar un senyal sense perdre informació, la freqüència de mostreig ha de ser, com a mínim, el doble de la freqüència màxima del senyal original (vegeu la fórmula 2).

$$f_s > 2 \cdot f_{max} \tag{2}$$

Si no es compleix aquest teorema, pot aparèixer un error de distorsió anomenat **aliàsing**. Es recomana veure el vídeo de la referència [11] per acabar d'entendre el concepte d'aliàsing i el teorema de Nyquist-Shannon.

Un cop digitalitzat i a l'ordinador, el senyal es pot tractar d'una forma molt més eficient i segura. Els senyals digitals són especialment robusts contra el soroll i les interferències.

[2] Idealment. Al món real existeixen interferències, distorsions no lineals i altres conceptes que poden afectar el senyal.

[3] No és objecte d'aquest treball explicar els avantatges i inconvenients de les dues menes.

[4] Al document només es fa referència el procés de transformació d'analògic a digital, ja que és l'únic que tractarà en aquest treball.

2.2 Arquitectures de captura de senyal

Hi ha diferents formes (arquitectures) per les que es pot dissenyar un transmissor/receptor de senyal. En aquesta subsecció s'exploren dues d'aquestes arquitectures: la superheterodina, la més feta servir; i la passa banda, una arquitectura més complexa i amb la que tracta aquest treball.

2.2.1 Arquitectura superheterodina

L'arquitectura superheterodina és el disseny fonamental que ha dominat la recepció de ràdio i la comunicació sense fils des de 1918.

El seu objectiu principal és modular el senyal rebut de radiofreqüència (RF) a una freqüència més baixa i fixa, anomenada Freqüència Intermèdia (FI), on és molt més fàcil i eficient processar el senyal, filtrar el soroll i amplificar-lo abans de mostrejar-lo.

2.2.2 Arquitectura passa-banda

L'arquitectura passa-banda representa un canvi de paradigma respecte a la superheterodina. En lloc de dependre d'etapes analògiques de mescla i modulació (heterodinatge) per baixar el senyal de freqüència abans de digitalitzar-lo, aquesta arquitectura realitza el mostreig directe d'un senyal situat en una banda qualsevol.

La clau d'aquest procés recau a fer un ús molt específic del teorema de Nyquist-Shannon i de l'aliàsing del qual tant ens adverteix. Si fem memòria, aquest teorema es regeix per la fórmula de l'equació 2. En aquesta equació, però, suposa un senyal on $f_{min} = 0$Hz (i.e. senyal de banda base). Si es generalitza el teorema per a senyals que es troben desplaçats en l'espectre, podem definir la condició mínima com:

$$f_s > 2 \cdot B \quad \text{on} \quad B = f_{max} - f_{min} \tag{3}$$

Tanmateix, cal anar amb molt de compte: l'equació 3 és una condició necessària però no suficient. No qualsevol freqüència superior a 2B és vàlida, ja que les rèpliques espectrals continuen existint i si no es tria la f_s de forma adequada es podrien superposar senyals, generant aliàsing. Ara bé, existeixen uns intervals concrets de freqüències permeses on aquestes rèpliques esdevenen els nostres aliats.

Si es pren per exemple el cas de la figura 2 (extret de [7]), on es disposa un senyal de $B = 5MHz$ a $f_c = 20$MHz. Si s'aplica Nyquist tal com a l'equació 2 s'hauria de mostrejar a $f_s = 45$MHz, ara bé, si es mostreja a $f_s = 17.5$MHz (una freqüència per sota els límits de Nyquist teòrics) es podrà veure com apareixen un parell de rèpliques a una freqüència inferior, tal com prediu Nyquist, però tampoc molesten, doncs no hi ha aliàsing.

Per comprovar si en un cas donat hi haurà aliàsing o no ens regim per l'equació 4. On m és el nombre de rèpliques que volem entre 0Hz i el senyal original.

$$\frac{2f_c - B}{m} \geq f_s \geq \frac{2f_c + B}{m} \tag{4}$$

Al gràfic de la figura 3 es pot veure representada aquesta relació entre l'aliàsing, l'amplada de banda i la freqüència de mostreig. El gràfic ha sigut normalitzat respecte de l'amplada de banda del senyal. Notem que apareix aliàsing per totes aquelles freqüències menors de dues vegades l'amplada de banda i per les bandes fora dels límits establerts per

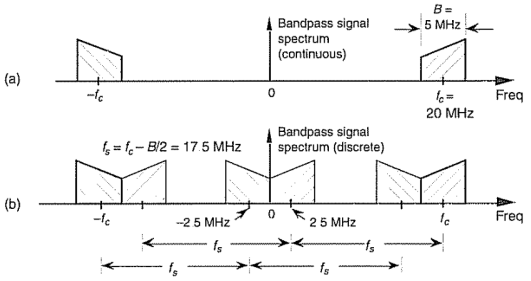


Fig. 2: Exemple del mostreig d'un senyal amb una freqüència de mostreig molt baixa. Font: [7]

l'equació 4. A la pràctica és recomanable no aproximar-nos massa als límits de les zones prohibides.

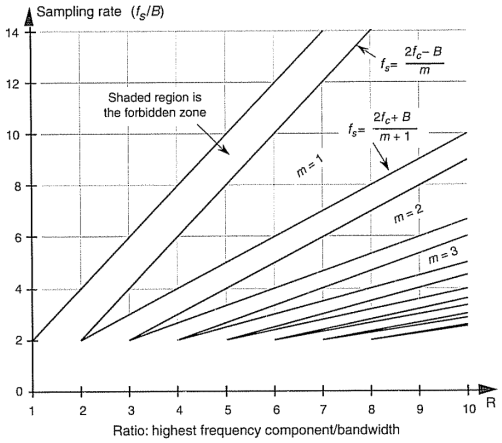


Fig. 3: Zones de captura permeses aplicant les equacions 4. Font: [7]

En conclusió, aquesta arquitectura resulta molt més complicada d'entendre i de dissenyar (en comparació amb la superheterodina), però resulta altament beneficiosa per a simplificar la cadena de hardware analògic, eliminant els mescladors i desplaçant tota la lògica de filtratge i desmodulació al domini del software.

3 Estat de l'Art

AQUESTA secció analitza detalladament el dispositiu central del projecte: la plataforma SDR USRP X440 d'Ettus Research (National Instruments)[5].



Fig. 4: Unitat USRP X440 utilitzada al llarg del treball

Ettus desenvolupa una gran varietat de USRPs diferents[5], entre elles la USRP X440. A diferència d'altres models, com la X410 o les X300s, la X440 representa un salt evolutiu important, ja que és la primera de la sèrie dissenyada específicament per a aplicacions de mostreig directe en radiofreqüència (*Direct RF*).

3.1 El mostreig directe (*Direct RF*)

Històricament, els convertidors ADC/DAC no eren prou potents per llegir senyals a GHz. Per tant, era necessari emprar una arquitectura superheterodina per realitzar una conversió a una freqüència intermèdia (f_i), on el tractament digital ja era viable. No obstant això, aquest enfocament comporta diversos desavantatges, com ara:

- **Complexitat del *hardware*:** Necessitat de múltiples etapes de mescla, filtres de rebuig d'imatge i oscil·ladors locals (LO) analògics.
- **Degradació del senyal:** Introducció de soroll de fase, desequilibris d'I/Q i DC offset originats pels components analògics.
- **Limitació de banda:** L'amplada de banda instantània queda restringida per les característiques físiques dels filtres de freqüència intermèdia.

Els models X440, però, poden operar amb freqüències de mostreig de fins a 4GHz, eliminant la necessitat de passos de mescla analògica i permetent el tractament del senyal directament en la banda de radiofreqüència (arquitectura passa-banda).

3.2 Integració

Una de les característiques més rellevants de la USRP X440 és la integració de tres subsistemes que tradicionalment requerien xips independents en un de sol: el SoC (*System on Chip*) **Xilinx Zynq UltraScale+**. Aquesta arquitectura, coneguda com a RFSoC, integra:

- **La Unitat de Processament (PS):** Basada en una arquitectura ARM, és l'encarregada d'executar un sistema operatiu per a tasques de control del dispositiu com ara les connexions amb la xarxa o l'execució de codi d'alt nivell.
- **La FPGA:** Permet una paral·lelització massiva que ajuda en gran manera a les tasques de processament de dades pesades en temps real. Aquesta FPGA ha de ser carregada prèviament amb una imatge (i.e. programa a executar)[5]. Addicionalment, aquesta FPGA queda dividida en dues meitats, anomenades daughterboards, on cadascuna actuarà com a frontera entre el xip i els senyals analògics.
- **Els Convertidors ADC/DAC:** Integrats directament en el mateix encapsulat, transformen els senyals analògics en digitals i al revés.

3.3 Característiques generals

Un cop analitzades les propietats arquitectòniques més importants de la X440, convé detallar les seves característiques generals principals:

- **Daughter Boards (DBs):** Disposa de dues DBs.

[5]A la taula 1 es detallen totes les imatges disponibles per a la USRP X440

Flavour	Chan.	Bandwidth per Channel	QSFP28 Port 0	QSFP28 Port 1	DDC DUC	DRAM
X4_200	8	200 MHz	4x 10 GbE	Unused	Yes	No
X4_400	8	400 MHz	4x 10 GbE	Unused	No	Yes
CG_400	8	400 MHz	100 GbE	100 GbE	No	No
X4_1600	2	1600 MHz	4x 10 GbE	Unused	No	Yes
CG_1600	2	1600 MHz	100 GbE	100 GbE	No	No

TAULA 1: DIFERENTS IMATGES PER LA FPGA DE LA USRP X440 PROPORCIONADES PER ETTUS.[6].

- **Nombre de canals:** Disposa de **8 canals** de recepció (RX) i 8 canals de transmissió (TX) totalment independents i sincronitzats. La meitat dels canals es troben a la primera DB i l'altra meitat a la segona.
- **Freqüència de mostreig (Sample Rate):** Els convertidors ADC poden treballar fins a 4**GHz** (Gigamostres per segon), mentre que els DAC arriben fins als 6.4GHz.
- **Resolució:** Utilitza convertidors de 14 bits, oferint un gran equilibri entre velocitat i precisió dinàmica.
- **Bytes per mostra:** Tot i tenir una resolució de 14 bits, aquests s'arrodoneixen a 16 bits per poder tractar amb un nombre sencer de bytes (16 bits = 2 Bytes). A l'hora de fer els càlculs s'haurà de recordar que aquest número s'ha de multiplicar per 2 (**4 Bytes per mostra** en total), per tindre en compte tant la fase com la quadratura.
- **Amplada de banda instantània:** Pot gestionar fins a 1.6GHz d'amplada de banda per canal.
- **Tensions límit (*Input Power*):** El rang d'entrada sol estar optimitzat per a senyals de poca potència. El nivell màxim abans de danyar els convertidors és de 13dBm en el pitjor dels casos.
- **Connectors QSFP28:** El dispositiu disposa de dos connectors QSFP28, els quals estan capacitats per a emetre i rebre fins a 100GbE[6].

3.4 La llibreria UHD

L'UHD (o *USRP Hardware Driver*) un controlador de codi obert i multiplataforma que proporciona una interfície de programació (API) i un conjunt d'eines de terminal per interaccionar amb els dispositius SDR d'Ettus (inclosa la X440).

La funció principal de l'UHD és abstreure la complexitat del *hardware*, cosa que permet que l'usuari controli paràmetres com la freqüència de sintonia, el guany o la taxa de mostreig sense haver de conèixer els detalls interns dels registres de la FPGA.

Podem dividir la suite d'eines que ofereix UHD en dues categories diferents:

- **Comandes** a la terminal: Aquestes són les més útils per interactuar amb la mateixa USRP. Aquestes comandes ens permeten fer coses com: descobrir dispositius i mirar característiques generals, canviar les imatges de les FPGAs o executar captures senzilles [3].

[6]Un Giga-Bit-Ethernet (GbE) és el mateix que 1Gb/s

- **Llibreries** per a llenguatges de programació C, C++ i Python. Aquestes llibreries estan fetes pensant a fer scripts de captura i facilitar la feina als usuaris de les USRPS[1][4].

4 METODOLOGIA

AQUEST és l'apartat principal del treball i on quedarà resumida la feina feta durant la duració d'aquest. Concretament es contemplen els temes següents:

- El procés de **familiarització** amb la USRP X440.
- Descripció del **procés de captura de senyals** amb la USRP X440.
- Explicació detallada dels **algorismes**, **interfícies** i **programes**.
- Descripció dels diferents **escenaris de prova**.

4.1 Familiaritzant-nos amb la USRP x440

La USRP x440 d'Ettus és un dispositiu molt punter i nou al mercat, tal com s'explica a l'estat de l'art del treball, i, per tant, tractar amb ella ha resultat ser una tasca més feixuga de l'esperat.

4.1.1 Connexions, descobriments i primers accessos

Abans de poder accedir al dispositiu, ha calgut connectar la USRP **al corrent,** a la **xarxa de la UAB**, a una **xarxa privada** directa entre l'ordinador i el dispositiu per via de dos cables QSDP28 de 100GbE i pel **Serial**.

Per tal de comprovar que la USRP està disponible des del nostre ordinador cal executar la comanda `uhd_find_devices` de la *suite* d'eines d'UHD. No cal dir que en el nostre primer intent el dispositiu no només no es mostrava "disponible", sinó que tampoc hi havia forma d'esbrinar la seva adreça IP. Per a això ha calgut connectar-nos al sistema operatiu aparell per via Serial i, des d'allà, executar la comanda `ifconfig` per esbrinar l'adreça IP.

Solucionat aquest problema la USRP ja ens surt com a "disponible"per a poder interactuar amb ella.

4.1.2 Actualitzant la FPGA i el sistema de fitxers

La documentació oficial d'Ettus[5] ens recomana carregar una imatge a la FPGA (vegeu la taula d'imatges a la taula 1) per a fer captures.

Això, però, tampoc ha resultat senzill a causa de diferents errors com ara: error de versions de la llibreria UHD, error amb el rellotge del sistema i errors de validació de certificats digitals entre d'altres.

Un cop solucionats tots els problemes[7] ja es poden (teòricament) realitzar captures de senyal.

[7]N. de l'A.: Per motius d'espai, en aquest informe no es detalla la resolució de cada error individual.

4.2 Procés de captura d'un senyal amb la USRP X440

Aquesta segona fase del treball se centra a estudiar a fons la forma amb la qual la USRP X440 captura senyal. Aquest procés s'explica seguint un ordre similar al que prendria un senyal entrant a la USRP (vegeu la figura 5). De totes maneres és molt recomanable llegir la documentació oficial: [2], a on s'entra en molts més detalls i casos d'ús que aquí.

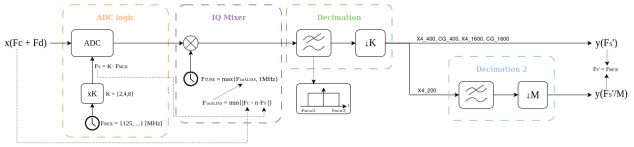


Fig. 5: Camí simplificat del senyal a través de la USRP.

Quan un senyal entra a la USRP X440 el primer que es troba és un ADC. Aquest ADC, tal com s'explica a l'estat de l'art del treball, és extremadament ràpid i ens permet estalviar-nos una etapa superheterodina prèvia.

La gràcia de l'ADC és que no només mostreja el senyal, sinó que, en fer-ho, crea diferents rèpliques del senyal a les diferents zones de Nyquist determinades per la freqüència de mostreig. Aquesta freqüència de mostreig no pot ser qualsevol, sinó que ha de ser dues, quatre o vuit vegades el valor d'una de les freqüències de rellotge preestablertes (vegeu la taula 3 de l'annex).

Un cop digitalitzades i creades les rèpliques, la USRP X440 modularà la rèplica més propera a banda-base per tal de centrar-la a 0Hz. Aquest procés és automàtic, nosaltres només cal que diguem la freqüència del senyal desitjat (a una zona de Nyquist qualsevol) i ja s'encarregarà el dispositiu de fer els càlculs necessaris per centrar la rèplica correcta.

Finalment, es fa un delmat del senyal per baixar el senyal de la freqüència de mostreig a la freqüència del rellotge de la FPGA, la qual serà dues, quatre o vuit vegades menys que la de mostreig.

Pel cas específic de la imatge de la FPGA X4_200 es pot afegir una fase de delmat addicional si es considera que les freqüències de rellotge són molt altes per l'aplicació en concret. També es pot fer servir aquesta etapa per acotar encara més l'amplada de banda de la resposta.

4.2.1 Coses a tindre en compte

Algunes de les coses més importants que cal tindre en compte a l'hora d'entendre el funcionament de la USRP X440 són les següents:

- A l'hora de calcular les zones de Nyquist el nostre senyal ha de caber completament dins del 80% de la zona de Nyquist, és a dir, deixant un marge del 10% de la zona a cada cantó. També s'inclou la part baixa de la 1a zona de Nyquist (i.e. banda base).
- La fase de mostreig de la rèplica més propera a zero serà de mínim 1MHz, generant d'aquesta forma un senyal complex en fase i quadratura.
- La imatge X4_200 de la FPGA només pot fer servir un conjunt de freqüències de rellotge molt limitades,

de forma que, per a captures d'amplades de banda superiors a 200MHz, es recomanarà fer servir imatges superiors i assumir que la freqüència de rellotge serà la que sigui[8]

4.2.2 Captura *dual-rate*

Per a senyals amb una amplada de banda molt gran potser no existeix una freqüència de mostreig que englobi la totalitat del senyal en una única zona de Nyquist. És per això que la USRP X440 ens permet capturar senyals amb dues freqüències de mostreig[9] diferents, una per cada daughterboard (vegeu la figura 6).

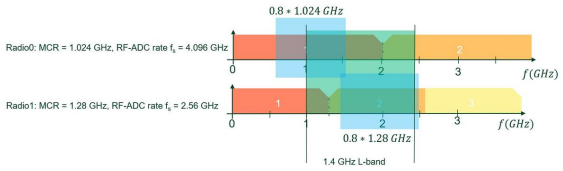


Fig. 6: Exemple de captura amb dual rate
. Font: [2]

4.3 Desenvolupament del programari

En aquest subapartat s'explicarà el disseny i implementació de tot el programari de captura i processat que s'ha dut a terme durant aquest treball.

4.3.1 Algorisme de generació d'opcions de captura

L'algorisme de generació d'opcions de captura és un seguit algorismes que prenen per entrades la freqüència central del senyal a capturar i l'amplada de banda d'aquest; i generen un seguit de llistats dels quals s'extrauran totes les opcions de captura possibles. Aquests llistats són:

1. Llistat de totes les **opcions parcials** possibles per les quals es pot capturar una part de (o tota) la banda d'interès.

2. Llistat de totes les **combinacions d'opcions parcials** per les quals s'aconsegueix capturar el total de la banda d'interès.

3. Llistat d'**opcions completes filtrades** i ordenades.

El primer algorisme és el més complex de tots, itera per totes les freqüències de mostreig de la taula 3, en calculen les zones de Nyquist i mira en quines entra el senyal desitjat, deixant un 10% de marge a cada banda de la zona de Nyquist. Cada freqüència vàlida es guarda en un fitxer JSON amb algunes metadades addicionals.

A partir de les opcions parcials generades pel primer algorisme, el segon es dedica a iterar per cadascuna d'aquestes opcions i genera totes les combinacions que permetin capturar la banda completa.

El tercer algorisme no és un algorisme per se[10], sinó que agafa totes les opcions completes i en filtra aquelles que no siguin vàlides ja sigui perquè l'arquitectura de la USRP no les permet o perquè l'usuari ha indicat que s'ha de complir alguna condició.

Per entendre el funcionament d'aquests algorismes a fons, es recomana mirar els codis directament. Aquests els trobareu o a l'annex d'aquest document (vegeu: 1 i 2) o al dossier adjunt al TFG.

4.3.2 Interfície gràfica per la captura de senyal

Un podria fer servir els scripts comentats anteriorment per buscar les opcions desitjades i executar comandes a partir d'elles, però com que un dels objectius del treball és facilitar al màxim la tasca de captura als futurs usuaris de la USRP, s'ha decidit implementar una interfície d'usuari (UI) que permeti a aquests interactuar amb el dispositiu a un nivell d'abstracció superior.

Aquesta interfície s'ha desenvolupat fent servir el framework de javascript VUE per al front-end i amb Python (amb *Flask*) per al back-end.

Quan un usuari entri per primer cop a la UI es trobarà amb una pestanya en blanc on se'ns demanarà que introduïm la freqüència central a la qual es pretén capturar i l'amplada de banda d'aquesta.

Donades aquestes dades es pot prémer el botó de "*Generate Options*", aquest botó comprovarà la validesa de les dades, les arreglarà i les enviarà als algorismes generadors d'opcions. Un cop generades es mostraran en un llistat just a sota del botó (vegeu la figura 7). També es poden afegir filtres indicant el nombre de canals que desitjats o l'ordre amb què es vulgui que surtin les opcions al llistat posterior.

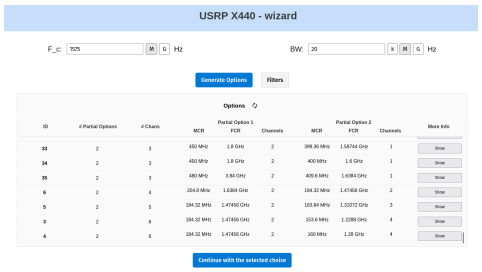


Fig. 7: Pestanya principal de la UI.

Addicionalment, s'ha afegit una mecànica per la qual l'usuari podrà veure més informació sobre l'opció de la captura a realitzar en una nova pestanya. En aquesta pestanya, a part de metadades, quedarà il·lustrat gràficament com queda el senyal en un diagrama en el domini freqüencial. Es podran veure tant la suma dels diferents canals com les zones de Nyquist de cada canal en qüestió (vegeu la figura 8).

Un cop seleccionada l'opció desitjada es passa a una pestanya de configuració de hardware, on s'indicaran a quins ports es troben connectades les antenes i quines connexions a la xarxa hi ha configurades (vegeu la figura 9). A més, per a opcions amb una amplada de banda inferior a 200MHz per cada canal, es podrà modificar la freqüència de mostreig.

Després de configurar el hardware ja es pot prémer el botó de "*Capture*". Aquest botó enviarà les dades de la captura configurades per l'usuari al back-end de l'aplicació, el qual les emmagatzemarà en un fitxer .json i executarà el programa de captura.

[8] Sempre es pot fer un delmat extra a matlab.

[9] Podem canviar tant les freqüències de mostreig com el MCR.

[10] De fet el seu codi no s'inclou a l'annex del treball. Tot i que el podreu trobar al dossier adjunt.

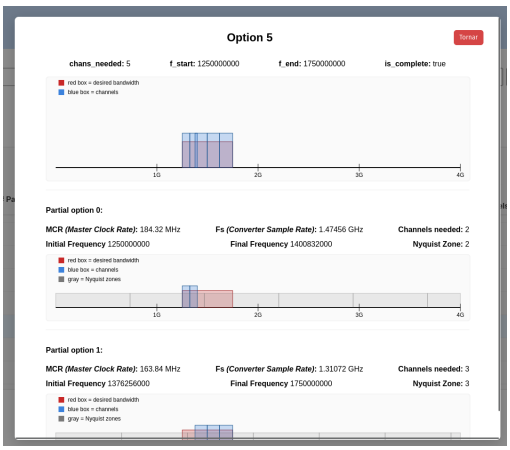


Fig. 8: Pestanya de mostra de dades d'una opció contreta.

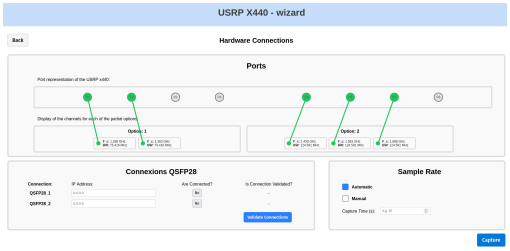


Fig. 9: Pestanya de configuració del hardware per a una opció concreta.

4.3.3 Script per la captura del senyal

En aquest subapartat s'ha desenvolupat un script que, donades les dades de captura del següent llistat, configura i executa una captura fent servir la USRP X440.

- **Adreces IP del dispositiu:** Introduïdes i validades per l'usuari. Com a mínim es necessita una i, en cas d'introduir-se'n dues es considerarà que l'usuari pretén fer una captura directa
- **MCRs i FCRs** de les diferents opcions. Provinents de les opcions parcials i diferenciades per a les dues daughterboards del dispositiu.
- **Freqüències centrals i canals:** Extretes a partir de la configuració HW de l'usuari i separades per daughterboard.
- **Freqüència de mostreig:** Introduïda per l'usuari.

Tots aquests paràmetres s'extreuen del fitxer JSON corresponent i es preparen per a fer-los servir al llarg del script [11].

El següent pas després d'extreure els paràmetres de captura és determinar la imatge de la FPGA que més ens convé. Aquesta tria és matemàtica i basada en els següents factors:

- Es triarà la imatge **X4_200** si el MCR més elevat és de 200MHz, l'usuari vol fer servir un SR personalitzat i la primera connexió QSFP es troba connectada i validada.
- Es triarà la imatge **X4_400** si el MCR més elevat és inferior a 512MHz i la primera connexió QSFP es troba connectada i validada.
- Es triarà la imatge **CG_400** si el MCR més elevat és inferior a 512MHz i les dues connexions QSFP es troben connectades i validades.
- Es triarà la imatge **X4_1600** si el MCR més baixet és superior a 1000MHz i la primera connexió QSFP es troba connectada i validada.
- Es triarà la imatge **CG_1600** si el MCR més baixet és superior a 1000MHz i les dues connexions QSFP es troben connectades i validades.

La tria d'una imatge de la FPGA o una altra tindrà conseqüències, tal com s'explica a l'estat de l'art del treball, però les úniques en ens afecten a l'hora de programar l'script de captura són:

- **Imatges amb DRAM**: Les imatges X4_400 i la X4_1600 disposen una DRAM de 8GiB per a poder emmagatzemar dades de captura abans d'enviar-les a l'ordinador principal. Això ens permet generar un flux de dades més controlat i no saturar els cables de xarxa a canvi d'una captura més petita (tot i que 8GiB per una captura sovint són suficients).
- **Freqüència de mostreig manual:** Per a la imatge X4_200 la freqüència de mostreig es pot introduir manualment gràcies als components DDC i DUC. Això ens permet alliberar-nos de les freqüències preestablertes per la USRP, sempre que aquestes no sobrepassin el límit forçat de 200MHz d'amplada de banda màxim per canal.

Tornant al script, després de configurar la FPGA per via de comandes a la terminal, el següent pas és indicar a la USRP quina arquitectura de captura volem. Per a això es farà ús de la llibreria d'UHD, la qual ens permet crear aquesta arquitectura en forma de graf. Els passos per crear aquesta arquitectura són:

1. **Crear un graf rfnoc buit** amb els arguments necessaris per a la nostra implementació (i.e. adreces IP, MCRs i FCRs).
2. **Crear i configurar els blocs de ràdio**. Aquests blocs són els més propers a la banda analògica i simbolitzen tot el procés de captura passa banda (Direct RF). A aquests blocs se'ls ha d'activar els ports pertinents de la USRP i assignar una freqüència de captura a cadascun. Els blocs de ràdio es configuren d'igual forma per a totes les imatges de la FPGA.
3. Després del bloc de ràdio s'haurà de posar un bloc o un altre depenent de la imatge de la FPGA triada. Aquests blocs poden ser:
   (a) Un **bloc replay** que s'encarregarà de controlar el flux entre les dades analògiques (ja digitalitzades) i les dades enviades a l'usuari. Aquest bloc només es pot posar per a aquelles imatges amb DRAM (i.e. X4_400 i X4_1600), de fet la seva funcionalitat recorda a aquella d'una DRAM.

[11] Vegeu més informació sobre aquests paràmetres i com afecten a l'us de la USRP X440 a l'estat de l'art del treball.

(b) Un **bloc DDC** que ens servirà per modificar la freqüència de mostreig en el cas de la imatge X4_200.

(c) Cap bloc. En el cas de les imatges on la transmissió de dades és directa i no podem canviar el SR (i.e. CG_400 i CG_1600) no cal posar cap bloc entremig.

4. Finalment, cal afegir un **bloc *streamer***, aquest servirà per enviar les dades que rebi (ja sigui del DUC, DRAM o Ràdio) per la xarxa.

Un cop es té tota l'arquitectura muntada es podrà validar amb una comanda proporcionada per la mateixa llibreria, aquesta ens retornarà o un missatge d'error o un de "triomf".

La penúltima etapa d'aquest procés consisteix a efectuar la captura i, de nou, ens topem amb diferències pel que fa als casos amb DRAM o sense. Ara bé, tant si es disposa de DRAM o no s'han de fer uns càlculs previs tant del nombre de mostres a rebre com de l'instant i duració de la captura de dades.

Fets els càlculs i en cas de disposar de DRAM, es configuren els receptors del graf per tal que capturin tots el mateix i de forma síncrona. Un cop preparat tot, ens esperem que la USRP realitzi la captura i després recuperem les dades a poc a poc fent servir *buffers* i blocs de dades de dimensions concretes.

Si no es disposa de DRAM el que es farà és crear un thread diferent per cada canal de recepció els quals, independentment, ompliran els *buffers* de dades a mesura que aquestes arribin.

Un cop es tenen els *buffers* de dades plens, es guardaran les dades en un fitxer al mateix directori que el script llest per ser processat.

4.4 Definició de les proves a realitzar

Aquesta darrera etapa del treball està enfocada a establir escenaris de prova i validació tant per al programari desenvolupat com per als objectius del treball.

Concretament, en aquesta secció es realitzarà una descripció tant de l'escenari com dels resultats esperats i dels objectius que es validarien en cas, el que no es tractarà són els resultats en si, aquests queden contemplats a l'apartat corresponent (5. Resultats).

4.4.1 Escenari 1: Captura de soroll

En aquest primer escenari l'objectiu no és capturar un senyal, sinó comprovar que tots els programaris desenvolupats funcionen de manera correcta, sense donar errors i que encaixen els uns amb els altres.

Es realitzarà doncs una captura a una freqüència aleatòria[12] amb tots els ports de la USRP desconnectats. A partir d'aquesta captura s'enviaran les dades recollides a un programa molt senzill que ens les mostrarà per pantalla en el domini temporal i freqüencial[13].

[12] A la pràctica s'ha fet servir la banda de GNSS: amb una $f_c = 1.575$GHz i una $BW = 20$MHz

[13] Aquest programa és una adaptació del programa d'un company del departament.

A la sortida s'hauria de veure soroll blanc, concretament, en domini temporal hauríem de veure un senyal aleatori i en el freqüencial una resposta plana.

4.4.2 Escenari 2: Captura d'un senyal sintètic

En aquest segon escenari es tracta la captura d'un senyal sintètic. Per sintètic ens referim a generar un senyal senzill a una freqüència i potència determinades. Això ens permetrà disposar una certesa i control absoluts sobre els paràmetres del senyal i el sistema, evitant molts factors externs que sovint dificulten la recepció i processat.

S'ha optat doncs per generar un senyal sinusoidal de $f_c = 120$MHz i 20mV_{pp} amb el generador de senyals (vegeu la figura 10) connectat a la USRP pel port RX0 de la DB0 i a un oscil·loscopi per poder prendre mesures de potència.

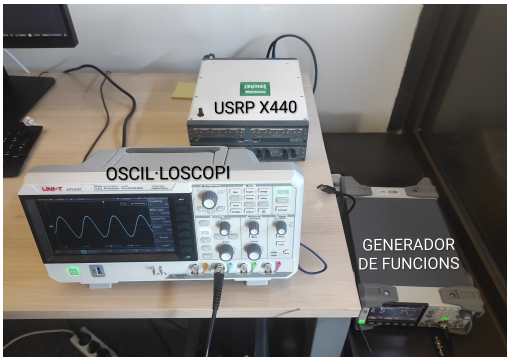


Fig. 10: *Setup* de l'escenari 2. Es mostren el generador de senyals, l'oscil·loscopi i la USRP X440

A la pràctica s'ha d'anar molt amb compte, ja que el límit de potència d'entrada de la USRP X440 és de 13dBm. En cada cas la potència es calcula d'una forma diferent, però per al nostre senyal sintètic la potència és de $P_{dBm} \approx -30$dBm, molt per sota de la potència límit però prou alt per escoltar-se per sobre del soroll.

Un cop processat el senyal s'espera veure un senyal sinusoidal complex al domini temporal i una delta (pic) al domini freqüencial.

Amb aquest escenari es pretenen validar els objectius número 1: Estudi i comprensió de l'arquitectura de la USRP X440; i 3: Crear una UI que faciliti la tasca de fer captures de senyal.

4.4.3 Escenari 3: Captura d'un senyal real

Finalment, es realitzarà una captura de senyal d'un satèl·lit LEO real. S'ha decidit capturar el senyal de la constel·lació Iridium pels següents motius:

- **Simplicitat de processament**: es pot veure el senyal només representant-lo en el domini freqüencial. Sense necessitat de cap postprocessament complex.

- **Cobertura satel·litària quasi constant:** La xarxa d'Iridium proporciona prou satèl·lits per a suposar que sempre n'hi ha un en el nostre rang de visió.

- **Hardware senzill:** El senyal cau al voltant dels 1.6GHz, amb una amplada de banda molt prima (<

10MHz). Això fa que es pugui connectar una antena "directament"al pot d'entrada, sense necessitat d'una arquitectura superheterodina entremig que afegeixi complexitat al sistema.

El setup de captura és el que podem veure a la figura 11 i queda compost per (ordre: analògic → digital):

1. **Antena** PN 100-00044-02 de Maxtena Int [8]: Aquesta antena s'ha penjat de la finestra del laboratori fent servir el suport pertinent.

2. **Amplificador** ZRL-2400LN de Mini-Circuits[9]: Aquest afegirà 24dB al senyal capturat, prou per fer que es vegi per sobre del soroll.

3. **DC-Blocker** DBK-89-S+ de Mini-Circuits[]: Aquest element de seguretat bloquejarà qualsevol corrent continu que s'escapi de l'amplificador cap a la USRP X440.

4. **USRP X440**.

Totes les connexions s'han realitzat amb els cables RF pertinents.

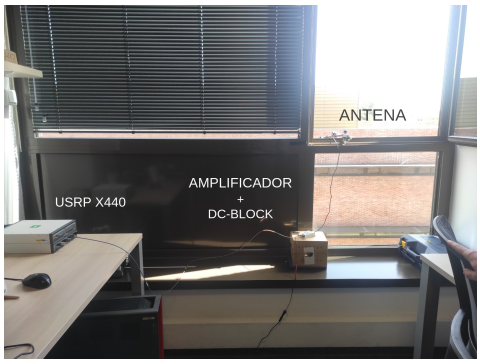


Fig. 11: *Setup* de l'escenari 3 amb tots els components indicats.

Visualment, s'ha de veure com el satèl·lit transmet paquets de dades de molt poca durada (mili-segons) i una amplada de banda de pocs kilohertzs a través de 5 canals diferents, aquests canals es troben centrats a les freqüències de la taula 2.

Channel	Frequency (MHz)
Quaternary	1626.10417
Tertiary	1626.14583
Ring Alert	1626.27083
Secondary	1626.39583
Primary	1626.4375

TAULA 2: TAULA DE FREQÜÈNCIES PER A CADA CANAL DE *Downlink* D'IRIDIUM.[10]

L'extracció d'uns resultats correctes a partir d'aquesta prova suposaria la validació de tots els objectius del treball. Addicionalment, en fer ús de la imatge X4_200 de la FPGA també es podrà validar l'ús correcte del component DDC, el qual fins ara no s'havia tingut en compte.

5 RESULTATS

En aquest darrer apartat s'avaluen els resultats extrets dels diferents escenaris de validació proposats a l'apartat 4.4.

5.1 Escenari 1: Captura de soroll

Aquest primer escenari és el que més cops s'ha hagut realitzar, degut a la naturalesa dels programes a no funcionar mai a la primera, però gràcies a factors com: missatges a la terminal (siguin d'errors o no), comprovacions manuals als fitxers .JSON o visualitzacions de les gràfiques temporals i freqüencials de les respostes; s'ha pogut solucionar tots els problemes i realitzar diverses execucions exitoses de l'escenari.

Amb l'èxit d'aquest escenari queda validada la integració dels diferents programes, algorismes i interfícies sense generar errors.

5.2 Escenari 2: Captura d'un senyal sintètic

S'ha dut a terme l'execució d'aquest segon escenari tal com es descriu a l'apartat 4.4.2. d'aquest treball i s'ha mostrat la resposta del senyal capturat per pantalla (vegeu la figura 12).

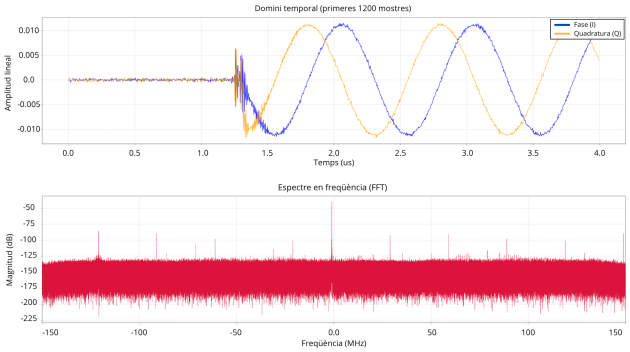


Fig. 12: Sortida de l'escenari 2: Captura d'un senyal sintètic.

A la figura 12 es veu clarament l'ona sinusoidal que es pretenia capturar, demostrant que, efectivament, s'està capturant senyal. Tot i que la captura és perfecte per a validar els objectius de l'escenari, cal fer esment dels següents fenòmens que es poden veure a la figura:

- **Senyal complex:** Tot i que el senyal generat és real la sortida és complexa (amb fase i quadratura) això és degut a la fase ÏQ mixer"(vegeu la figura 5 i la seva explicació l'apartat 4.2.1) que ens centra el senyal a capturar.

- **Banda base:** El senyal que es pot veure a la figura es troba en banda base (i.e: 0Hz)[14], de nou, això és degut al ÏQ mixer"que ens centra el senyal.

- **Deltes** fluixes **a altres freqüències**: Al llarg de la resposta freqüencial apareixen línies verticals menys potents que la principal. Aquestes formen part de les

[14] S'ha forçat un petit desplaçament respecte del zero per poder veure la resposta temporal de forma correcta, una captura fidel hagués aparegut a 0Hz.

rèpliques d'altres zones de Nyquist, les quals no s'han atenuat prou.

Aquesta captura és perfecta per demostrar tant el correcte funcionament del programa de captura com els objectius 1 i 3 d'aquest treball.

5.3 Escenari 3: Captura d'un senyal real

Aquest darrer escenari s'ha dut a terme seguint les instruccions de l'apartat 4.4.3 d'aquest treball i s'ha obtingut la resposta que queda representada a la figura 13.

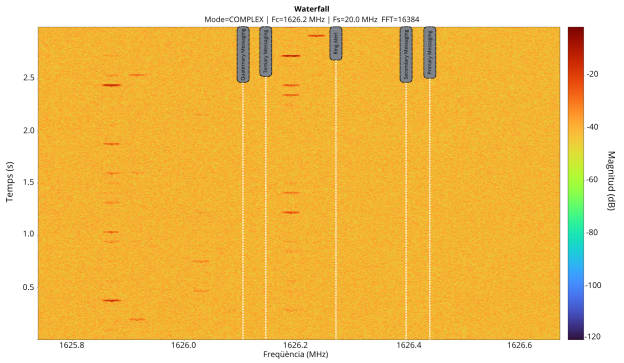


Fig. 13: Sortida de l'escenari 3: Captura d'un senyal real.

En aquesta resposta es pot veure clarament el comportament esperat i descrit a l'apartat 4.4.3. Concretament, queden molt ben representats els paquets de dades breus i d'amplada de banda de l'ordre de KHz distribuïts per l'espectre en forma de canals verticals espaiats.

L'única cosa que es pot argumentar en contra d'aquest resultat és el desfasament entre els canals que es poden veure la captura i els ideals (representats per línies discontínues a la mateixa figura). Aquest desfasament tot i semblar estrany, és un clar comportament de l'efecte Doppler, cal recordar que els satèl·lits LEO es desplacen a una velocitat altíssima i que, per tant, sempre s'ha de tindre en compte aquest efecte a l'hora de mesurar senyal real.

Addicionalment, aquesta captura s'ha realitzat a la 4a zona de Nyquist i per a la imatge X4_200, la qual disposa de DDC, demostrant encara més que s'ha comprès correctament el funcionament del dispositiu i que s'ha dissenyat un programari capaç de funcionar per a tots els escenaris. A efectes del treball queden validats tots els objectius establerts al capítol 1.

6 Conclusions

Aquest Treball de Final de Grau queda conclòs de forma satisfactòria, havent acomplert amb èxit tots els objectius proposats inicialment.

Al llarg del projecte s'ha demostrat una correcta comprensió del dispositiu USRP X440 i de la seva arquitectura de mostreig directe (Direct RF), superant la complexitat que presenta la seva configuració.

Així mateix, s'ha validat el correcte funcionament i la integració dels diferents programaris, algorismes i interfícies desenvolupats; els quals no només assoleixen l'objectiu d'abstraure i simplificar el procés de captura de senyals, sinó que també es poden fer servir com a base de futurs projectes del departament.

Finalment, les proves realitzades al laboratori, tant per a senyals sintètics com reals, confirmen la viabilitat i robustesa de la solució implementada, deixant una estació de seguiment totalment operativa i accessible per a futurs usuaris del departament.

Personalment, aquest treball ha sigut molt gratificant i una gran cloenda al que han sigut 5 anys d'estudis de dues matèries tan diferents, però que, sovint, es complementen de forma tan meravellosa com en aquest treball.

7 Ús de la IA generativa

S'ha fet ús de la IA generativa per a la realització d'aquest informe. Aquest ús, però, ha sigut mesurat, tal com se'ns ha demanat a nosaltres (estudiants). És usos principals de la IA es resumeixen en:

- Assistent de programació, ajudant en tasques com moure funcions de fitxers, creant nous components de VUE a partir de components existents o programant funcions bàsiques i que s'haguessin copiat d'igual forma d'internet (p.e: lectura/escriptura de JSONs o enviar/rebre dades per POST/GET en js).
- Assistència en la redacció de l'informe en forma d'opinions sobre l'escrit, mai com a redactor. El mateix es pot dir de la usabilitat de la pàgina web.
- Assistència en format del LaTeX, en especial a l'hora de fer taules i lísings.
- Generació d'imatges. A causa de les mancances artístiques de l'estudiant, s'ha demanat a la IA que generi la imatge 1. a partir d'un esbós.

No s'ha realitzat en aquest treball tasques per a cap aspecte del treball sotmès a avaluació, com ara per a generació d'idees o d'extracció de conclusions, per mitjà de la IA.

Agraïments

M'agradaria agrair tant a en José A. Lopez Salcedo pel seu acompanyament i mentoratge al llarg del treball com a en Toni Reus Bergas pel seu compromís i gran ajuda.

Referències

[1] Marc Anastasopoulos. *PySDR: A Guide to SDR and DSP using Python – USRP*. [Tutorial en línia; accedit el 23-maig-2026]. URL: %5Curl%7Bhttps://pysdr.org/content/usrp.html%7D.

[2] Ettus Research. *About Sampling Rates and Master Clock Rates for the USRP X440 – Option 3: MCR 512 MHz, RF-ADC converter rate = 4.096 GHz, 3 channels*. [En línia; accedit el 23-maig-2026]. National Instruments. URL: %5Curl%7Bhttps://kb.ettus.com/About_Sampling_Rates_and_Master_Clock_Rates_for_the_USRP_X440#Option_3:_MCR_512_MHz.2C_RF-ADC_converter_rate_.3D_4.096_GHz.2C_3_channels%7D.

[3] Ettus Research. *UHD - Device Driver Usage*. [Manual en línia; accedit el 23-maig-2026]. National Instruments. URL: %5Curl%7Bhttps://files.ettus.com/manual/page_driver_usage.html%7D.

[4] Ettus Research. *UHD Python API*. [Manual en línia; accedit el 23-maig-2026]. National Instruments. URL: %5Curl%7Bhttps://files.ettus.com/manual/page_python.html%7D.

[5] Ettus Research. *USRP X440 Knowledge Base*. [En línia; accedit el 23-maig-2026]. National Instruments. URL: %5Curl%7Bhttps://kb.ettus.com/X440%7D.

[6] Ettus Research. *USRP X4xx FPGA images*. [En línia; accedit el 14-juny-2026]. National Instruments. URL: %5Curl%7Bhttps://files.ettus.com/manual/page_usrp_x4xx.html#x4xx_updating_fpga_types%7D.

[7] Richard G. Lyons. *Understanding Digital Signal Processing*. 3rd. Upper Saddle River, NJ: Prentice Hall, 2010. ISBN: 978-0137027415.

[8] Maxtena. *M1621HCT-EXT2 (100-00044-02) Datasheet: Helical Iridium Certified Passive External Magnet Mount Antenna*. [Datasheet en línia; accedit el 25-juny-2026]. URL: %5Curl%7Bhttps://www.alldatasheet.com/datasheet-pdf/pdf/1360934/MAXTENA/100-00044-02.html%7D.

[9] Mini-Circuits. *ZRL-2400LN Datasheet: Low Noise Amplifier 50 Ohm 1000 to 2400 MHz*. [Datasheet en línia; accedit el 25-juny-2026]. URL: %5Curl%7Bhttps://www.alldatasheet.com/datasheet-pdf/pdf/94680/MINI/ZRL-2400LN.html%7D.

[10] SigIDWiki contributors. *Iridium – Signal Identification Wiki*. [En línia; accedit el 25-juny-2026]. URL: %5Curl%7Bhttps://www.sigidwiki.com/wiki/Iridium%7D.

[11] Zach Star. *The intuition behind the Nyquist-Shannon Sampling Theorem*. [Vídeo de YouTube; accedit el 23-maig-2026]. 2024. URL: %5Curl%7Bhttps://www.youtube.com/watch?v=Jv5FU8oUWEY%7D.

8 ANNEX

MCR (MHz)	RF-ADC Converter Rates (GHz)	MCR (MHz)	RF-ADC Converter Rates (GHz)
125.00	1.0	...	...
128.00	1.024	983.04	1.96608, 3.93216
133.12	1.06496	998.40	3.9936, 1.9968
150.00	1.2	1000.00	4.0, 2.0
153.60	1.2288	1024.00	4.096, 2.048
160.00	1.28	1050.00	2.1
163.84	1.31072	1064.96	2.12992
184.32	1.47456	1075.20	2.1504
199.68	1.59744	1080.00	2.16
200.00	1.6	1105.92	2.21184
204.80	1.6384	1120.00	2.24
240.00	1.92	1125.00	2.25
245.76	1.96608	1146.88	2.29376
250.00	2.0, 1.0	1152.00	2.304
256.00	2.048, 1.024	1198.08	2.39616
266.24	2.12992, 1.06496	1200.00	2.4
300.00	2.4, 1.2	1228.80	2.4576
307.20	2.4576, 1.2288	1250.00	2.5
320.00	2.56, 1.28	1280.00	2.56
327.68	2.62144, 1.31072	1290.24	2.58048
360.00	2.88, 1.44	1310.72	2.62144
368.64	1.47456, 2.94912	1331.20	2.6624
375.00	3.0, 1.5	1350.00	2.7
384.00	1.536, 3.072	1382.40	2.7648
399.36	3.19488, 1.59744	1397.76	2.79552
400.00	3.2, 1.6	1400.00	2.8
409.60	3.2768, 1.6384	1433.60	2.8672
450.00	1.8, 3.6	1440.00	2.88
460.80	1.8432, 3.6864	1474.56	2.94912
480.00	3.84, 1.92	1500.00	3.0
491.52	3.93216, 1.96608	1536.00	3.072
500.00	4.0, 2.0, 1.0	1600.00	3.2
512.00	1.024, 2.048, 4.096	1625.00	3.25
532.48	1.06496, 2.12992	1638.40	3.2768
552.96	2.21184, 1.10592	1650.00	3.3
599.04	1.19808, 2.39616	1658.88	3.31776
600.00	1.2, 2.4	1664.00	3.328
614.40	1.2288, 2.4576	1680.00	3.36
625.00	1.25, 2.5	1689.60	3.3792
640.00	2.56, 1.28	1720.32	3.44064
655.36	1.31072, 2.62144	1730.56	3.46112
665.60	1.3312, 2.6624	1750.00	3.5
720.00	1.44, 2.88	1760.00	3.52
737.28	1.47456, 2.94912	1792.00	3.584
750.00	1.5, 3.0	1797.12	3.59424
768.00	1.536, 3.072	1800.00	3.6
798.72	3.19488, 1.59744	1802.24	3.60448
800.00	1.6, 3.2	1843.20	3.6864
819.20	3.2768, 1.6384	1863.68	3.72736
840.00	1.68, 3.36	1875.00	3.75
860.16	3.44064, 1.72032	1920.00	3.84
875.00	3.5, 1.75	1950.00	3.9
896.00	3.584, 1.792	1966.08	3.93216
900.00	1.8, 3.6	1996.80	3.9936
921.60	1.8432, 3.6864	2000.00	4.0
931.84	1.86368, 3.72736	2027.52	4.05504
960.00	3.84, 1.92	2048.00	4.096
...	...		

TAULA 3: TAULA DE MCRS I FREQÜÈNCIES DE CONVERSIÓ DE DADES DE LA USRP X440

Listing 1: Funció per generar opcions parcials

```
# Generate Options Functions
def generatePartialOptions(f_min, f_max, mcr_converter_rates_table_path,
    partial_options_path):
    # Create the array for the partialOptions
    partial_options = []

    # Read mcr_converter_rates table JSON file
    mcr_converter_rates_table = readJSON(mcr_converter_rates_table_path)
    if mcr_converter_rates_table is None:
        print("Error: No s'ha pogut carregar la taula de MCR i FCR des del fitxer JSON.
            ")
        return False

    # Iterate through every item (MCR) in the mcr_converter_rates table
    for entry in mcr_converter_rates_table:
        # Extract the MCR (Master Clock Rate) in Hz
        mcr_hz = entry["mcr_mhz"] * 1e6

        # Extract the maximum BW allowed by the MCR
        usable_bw_per_chan = 0.8 * mcr_hz

        # Iterate through all the possible converter rate frequencies
        for fcr_ghz in entry["rfdc_converter_rates_ghz"]:
            # Extract the FCR (Frequency Conversion Rate) in Hz
            fcr = fcr_ghz * 1e9

            # Calculate the width of the Nyquist zones
            nyquist_bw = fcr / 2

            # Set the Nyquist zone margins to 80% the Nyquist zone (10% on each side)
            margin = 0.1 * nyquist_bw

            # Iterate through zones (1st up to the 8th)
            current_f_bottom = f_min # Helper variable to keep track of the frequencies
            for zone_idx in range(1, 9):
                # Get the min and max frequencies of the current Nyquist zone
                zone_min = (zone_idx - 1) * nyquist_bw
                zone_max = zone_idx * nyquist_bw

                # Define the "Safe Zone" within this Nyquist window
                safe_min = zone_min + margin
                safe_max = zone_max - margin

                # Check if our target signal overlaps with this safe zone
                overlap_min = max(current_f_bottom, safe_min)
                overlap_max = min(f_max, safe_max)

                # If there is an overlap, we create a partial option for this zone
                if overlap_max > overlap_min:
                    interest_bw = overlap_max - overlap_min
                    chans_needed = math.ceil(interest_bw / usable_bw_per_chan)

                    option = {
                        "option_id": f"mcr{entry['mcr_mhz']}_fcr{fcr_ghz}_zone{zone_idx
                            }",
                        "mcr_mhz": entry["mcr_mhz"],
                        "fcr_ghz": fcr_ghz,
                        "nyquist_zone": zone_idx,
                        "f_start": overlap_min,
                        "f_end": overlap_max,
                        "chans_needed": chans_needed,
                        "is_complete": (overlap_min == f_min and overlap_max == f_max)
                    }
                    partial_options.append(option)

                    # Update progress
```

```
                    current_f_bottom = overlap_max

    # Store the partial options in a json file
    storeJSON(partial_options, partial_options_path)

    return True
```

Listing 2: Funció per generar opcions completes

```
def generateCompleteOptions(f_min, f_max, partial_options_path):
    complete_options = []
    partial_options = readJSON(partial_options_path)
    if partial_options is None:
        return False

    while len(partial_options) > 0:
        p_opt = partial_options.pop(0)
        if p_opt["is_complete"]:
            complete_options.append({
                "complete_option_id": len(complete_options),
                "partial_options": [p_opt],
                "f_start": p_opt["f_start"],
                "f_end": p_opt["f_end"],
                "chans_needed": p_opt["chans_needed"],
                "is_complete": True
            })
        else:
            combinations = [{"partial_options_list": [p_opt], "is_complete": False}]
            remaining = partial_options.copy()
            counter = 0
            while counter < len(remaining) and len(remaining) > 0:
                counter += 1
                curr = remaining.pop(0)
                if curr["is_complete"]: continue
                remaining.append(curr)

                for comb in combinations:
                    if comb["is_complete"]: continue
                    # Llogica d'expansio de combinacions
                    if (condicio_expansio_dreta):
                        comb["partial_options_list"].append(curr)
                    elif (condicio_expansio_esquerra):
                        comb["partial_options_list"].insert(0, curr)

                    if (comb_es_completa):
                        comb["is_complete"] = True

            # Filtre de resultats
            for comb in combinations:
                total_ch = sum([o["chans_needed"] for o in comb["partial_options_list"]])
                if comb["is_complete"] and total_ch <= 8:
                    complete_options.append({...})

    storeJSON(complete_options, './assistanceJSONs/completeOptions.json')
    return True
```